

김종록

BACKEND & AI APPLICATION DEVELOPER

010-7742-1623 | xhxhahs2@gmail.com | 경기 하남시

PROFILE

병원 전산 운영 3년 11개월 동안 여러 진료·행정부서의 장애에 대응하고 사용자 요청을 처리하며, 문제를 업무 흐름과 시스템 구조의 관점에서 파악하는 경험을 쌓았습니다.

이후 Spring Boot 기반 Backend와 React 화면을 구현하고, 머신러닝·딥러닝 모델 개발과 LLM 에이전트·RAG 챗봇 구현을 경험했습니다. 5인 팀 PL로 참여한 BidMatch에서는 AI 기능을 Backend 규칙과 검증·대체 처리로 보완하고 AI 자동화 테스트 242건을 통과했습니다.

운영 환경에 대한 이해를 바탕으로 AI 기능을 안정적인 Backend 서비스에 연결하고, 예외 상황에서도 사용자가 신뢰할 수 있는 결과를 제공하는 개발자로 성장하고 있습니다.

Portfolio | [GitHub](#) | [BidMatch Project](#) | [BidMatch Service](#)

CORE COMPETENCIES

- Spring Boot 기반 인증·회원·기업정보·고객센터·알림 API와 React 사용자 흐름 구현
- 정형 데이터 ML, 영상·시계열 DL 모델 개발과 LLM 에이전트·RAG 챗봇 구현
- BGE-M3 검색·Qwen 분류와 Spring 규칙 판정을 분리하고 검증·fallback 적용
- 5인 팀 프로젝트 리더(PL)로 API 계약, 기능 범위, 테스트와 시연 자료를 조율

TECHNICAL SKILLS

Backend	Java 17 · Spring Boot 3.5 · Spring Security · Spring Data JPA · MyBatis · SSE
AI · ML/DL	Python · FastAPI · scikit-learn · TensorFlow/Keras · PyTorch · YOLOv8 · RAG · BGE-M3 · Qwen/Ollama
Frontend	React 18 · Vite 5 · React Router · Axios · Streamlit
Data · Infra	PostgreSQL · pgvector · Redis · MariaDB · Supabase · Docker(실행·검증)
Test · Collaboration	JUnit 5 · pytest · Bruno · Git/GitHub · GitHub Actions · Jira · Slack

WORK EXPERIENCE

(주)시스게이트 · 한림대학교의료원(강동) 전산 운영 | 2019.08 – 2023.06 · OA 운영 및 사용자 지원

담당 업무

- 병원 내 여러 진료·행정부서의 OA 요구사항 수집, 시스템 구성 검토와 도입 지원
- 장애·사용자 요청 대응, 신규 시스템 교육 자료 제작과 사용자 교육 진행

경험 및 직무 연결점

- 반복 문의를 운영 매뉴얼·정책·Q&A 대응 체계로 정리하고 원격 OA 전환 지원
- 사용자 업무와 영향 범위를 고려해 원인을 좁히는 경험을 개발의 예외 처리·운영 관점으로 확장

EDUCATION

휴먼AI교육센터 · 심화_인공지능(AI) 서비스 기반 웹 개발자 심화 프로젝트 | 2026.06.11 – 2026.08.11

KG IT BANK · 핀테크 서비스를 위한 풀스택 개발자 양성 과정 | 2024.08.05 – 2025.02.13

여주대학교 · 컴퓨터정보과 | 2016.03 – 2020.02 · 졸업

AI EDUCATION HIGHLIGHTS

- scikit-learn 기반 데이터 전처리·모델 비교와 불균형 데이터 평가
- YOLOv8-LSTM 기반 객체 탐지, 이상탐지와 시계열 예측 모델 개발
- LLM Function Calling·RAG 챗봇과 응답 검증·fallback 구현
- Spring Boot·FastAPI·React 기반 Backend·AI·사용자 화면 연동

SELECTED PROJECTS

BidMatch · AI 기반 공공입찰 맞춤 추천·자가 자격 진단 서비스 | 2026.07.09 – 2026.08.07 · 5인 팀 PL

나라장터 공고와 기업정보를 연결해 맞춤 공고 추천, 근거 기반 자가 자격 진단, 알림과 고객센터를 제공하는 서비스

- Java 17·Spring Boot로 일반/기업회원 인증, 기업정보·면허·실적, 고객센터와 SSE 알림 API 구현
 - React 사용자 화면을 연동하고 Bruno 요청으로 Backend·AI API 계약과 예외 응답 검증
 - FAQ RAG 챗봇에 역할별 검색, 낮은 유사도 차단, 다중 의도, LLM fallback과 민감정보 마스킹 적용
 - BGE-M3 근거 검색과 Qwen 유형 분류 결과를 Spring 규칙 판정으로 재검증하는 자가 자격 진단 구조 설계
 - 병렬 개발 중 DB 마이그레이션(Flyway) 버전·체크섬·스키마 충돌을 기존 이력을 보존한 후속 마이그레이션으로 해결
 - 5개 저장소 118개 비병합 커밋 기여, AI 자동화 테스트 242건, Backend 테스트, 사용자·관리자 Frontend 빌드 통과
- 기술: Spring Boot · FastAPI · React · PostgreSQL · Redis · BGE-M3 · Qwen/Ollama · RAG · PyMuPDF · Docker(실행·검증)

지능형 교통 관제 시스템 | 2026.07.02 – 2026.07.07 · 개인

- YOLOv8 차량 탐지 결과와 Supabase 로그 조회를 LLM Function Calling으로 연결
- 날짜 환각과 도구 호출 오류를 시간 파서·DB 결과 검증·대체 모델 경로로 완화하고 Word 관제 일지 생성

기술: Python · YOLOv8 · Supabase · LLM Function Calling · Streamlit · python-docx

CCTV 교통량 이상탐지·예측 | 2026.06.25 – 2026.07.01 · 개인

- YOLOv8 탐지, LSTM Autoencoder 이상탐지, ITS CCTV, Supabase 로그를 Streamlit 화면으로 통합
- 입력 차원 불일치와 영상 지연을 reshape, 프레임 스킵, 표시 영상 리사이징으로 해결

기술: Python · YOLOv8 · TensorFlow/Keras · LSTM Autoencoder · Supabase · Streamlit

항공편 지연 예측 | 2026.06.18 – 2026.06.24 · 개인

- 약 25만 건 데이터를 전처리하고 XGBoost 등 모델을 비교해 불균형 데이터의 지연 Recall 중심으로 평가
- 전처리·모델을 scikit-learn Pipeline으로 패키징해 Streamlit에 연결, 프로젝트 보고서 기준 Recall 65%·ROC-AUC 0.785 기록

기술: Python · pandas · scikit-learn · XGBoost · Pipeline · Streamlit

맞춤 공고 자동 수집·메일 알림 프로토타입 | 2026.06.11 – 2026.06.17 · 팀 PL

- Flask·React로 OAuth/JWT 인증, Gmail 알림, Gemini 공고 요약, APScheduler 배치를 연결
- UTC/KST 발송 시간 차이를 Asia/Seoul 기준으로 보정하고 최종 Spring·FastAPI 프로젝트로 구조 확장

기술: Flask · React · OAuth/JWT · Gmail SMTP · Gemini · APScheduler

Spike · Spring Boot 기반 금융 서비스 | 2025.01 – 2025.02 · 팀 프로젝트

- Spring Security와 BCrypt를 적용한 회원가입·로그인, 역할별 접근 제어와 마이페이지 구현
- 관리자 사용자 CRUD·페이징과 보이스피싱 의심 계좌 신고·검토·상태 변경 흐름 구현

기술: Java 17 · Spring Boot 2.7 · JSP · JPA · MyBatis · Oracle DB

자기소개서

BACKEND & AI APPLICATION DEVELOPER

01 사용자의 문제에서 출발해, 안정적인 서비스로 연결하다

병원 전산실에서 근무하며 여러 진료·행정부서에서 발생하는 시스템 요청과 장애를 처리했습니다. 같은 기능도 사용자의 업무와 상황에 따라 전혀 다른 문제로 이어질 수 있었고, 기술적으로 정상인 시스템도 사용자가 이해하고 활용하지 못하면 좋은 서비스가 될 수 없다는 점을 배웠습니다.

반복적으로 발생하는 문의는 운영 매뉴얼과 교육 자료로 정리하고, 신규 시스템 도입과 개편 과정에서는 비IT 직군 사용자를 대상으로 교육을 진행했습니다. 특히 코로나19 기간에는 전자결재와 원격 OA 접근 체계 전환 과정에서 사용자 안내와 장애 대응을 담당했습니다. 이러한 경험을 통해 눈앞의 오류만 해결하는 것보다 문제가 발생한 원인과 사용자의 업무 흐름을 함께 이해하는 것이 중요하다는 사실을 알게 되었습니다.

업무를 수행하면서 시스템을 사용하는 사람을 지원하는 역할을 넘어, 시스템의 구조를 직접 설계하고 구현하고 싶다는 목표가 생겼습니다. 이를 계기로 Java와 Spring Boot 기반 풀스택 개발 과정을 이수했고, 이후 정형 데이터 머신러닝과 영상·시계열 딥러닝 모델을 개발하고 LLM 에이전트·RAG 챗봇을 구현했습니다.

최종 프로젝트인 BidMatch에서는 5인 팀에서 프로젝트 리더(PL)를 맡아 회원·기업 인증, 기업정보·면허·실적 관리, 고객센터와 실시간 알림 API를 구현했습니다. 또한 FAQ RAG 챗봇과 첨부문서 자가 자격 진단을 개발하며 Spring Boot 백엔드와 FastAPI AI 기능을 하나의 사용자 흐름으로 연결했습니다.

이러한 경험을 바탕으로 사용자의 문제를 이해하고, AI 기능을 안정적인 Backend 서비스에 연결하며 예외 상황에서도 신뢰할 수 있는 결과를 제공하는 개발자로 성장하고 있습니다.

02 문제를 임시로 덮지 않고 재발하지 않는 구조를 고민하다

BidMatch 개발 과정에서 여러 팀원이 동시에 데이터베이스(DB) 스키마를 변경하면서 Flyway 마이그레이션 버전 중복과 적용된 파일의 체크섬 불일치 문제가 발생했습니다. 일부 개발자의 환경에서는 정상적으로 실행됐지만, 다른 환경에서는 애플리케이션이 시작되지 않거나 기존 데이터베이스 구조와 최신 JPA 엔티티 정의가 일치하지 않았습니다.

이미 실행된 마이그레이션 파일을 수정하거나 개발자의 로컬 데이터베이스를 강제로 초기화하면 당장의 오류는 해결할 수 있었습니다. 그러나 이러한 방식은 다른 개발 환경이나 향후 배포 환경에서 같은 문제를 다시 발생시킬 가능성이 있었습니다.

목표는 팀원들의 기존 DB를 초기화하지 않으면서, 기존 DB와 새로 구성한 DB에서 모두 애플리케이션이 실행되도록 만드는 것이었습니다.

먼저 Git 이력과 적용된 마이그레이션 정보를 비교해 충돌이 발생한 시점과 원인을 확인했습니다. 이후 기존에 적용된 SQL은 변경하지 않고 새로운 버전의 후속 마이그레이션을 추가했습니다. 또한 이전 컬럼명과 최신 JPA 엔티티가 기대하는 컬럼명의 차이를 보정해 기존 데이터를 가진 DB와 새로 구성한 DB 모두에서 애플리케이션이 실행될 수 있도록 했습니다.

수정 후에는 Spring 애플리케이션이 정상적으로 실행되는지 확인하고 전체 백엔드 테스트도 다시 진행했습니다. 이 내용을 팀원들과 공유하고, 이후에는 마이그레이션 파일을 올리기 전에 버전과 파일 위치, 기존 DB 적용 여부를 먼저 확인하기로 했습니다.

이 경험을 통해 특정 개발 환경에서만 오류가 사라지는 것은 근본적인 해결이 아니라는 점을 배웠습니다. 기존 DB를 사용하는 팀원과 새로 환경을 만드는 사람 모두 정상적으로 실행할 수 있어야 해결된 것이라고 생각합니다.

03 AI의 답을 그대로 믿지 않고 검증 가능한 구조를 만들다

공공입찰 서비스에서 자격조건을 잘못 안내하면 사용자의 의사결정에 영향을 줄 수 있습니다. 따라서 BidMatch의 자가 자격 진단 기능을 구현할 때 LLM이 공고문 전체를 읽고 최종 결론을 내리도록 하지 않았습니다.

먼저 BGE-M3 임베딩 모델을 활용해 공고 첨부문서에서 자격과 관련된 근거 문장을 검색했습니다. 이후 LLM은 검색된 문장 중 자격 판단에 필요한 근거 ID를 선택하고, 해당 근거가 지역, 면허, 실적, 인력, 공동수급, 제출서류 중 어떤 유형인지 분류하도록 역할을 제한했습니다. 최종 충족 여부는 Spring Boot가 기업정보와 구조화된 공고 조건을 비교해 판정하도록 구성했습니다.

LLM이 존재하지 않는 근거를 만들거나 정해진 형식과 다른 응답을 반환하는 경우도 고려했습니다. LLM이 반환한 근거가 실제 원문에 존재하는지 다시 확인하고, 시간 초과나 모델 장애가 발생하면 규칙 기반 기본 진단으로 전환했습니다. 자동으로 판단하기 어려운 조건은 충족으로 단정하지 않고 사용자가 공고 원문을 확인하도록 안내했습니다.

FAQ RAG 챗봇에서도 유사도 점수가 낮은 질문을 억지로 기존 FAQ와 연결하지 않도록 차단 기준을 두었습니다. 사용자 역할에 따른 검색 범위와 여러 의도가 포함된 질문 처리를 적용하고, LLM 응답에 문제가 생기면 승인된 FAQ 답변을 사용하도록 했습니다. 이메일·전화번호·사업자번호와 같은 개인정보는 DB와 추적 도구에 저장되기 전에 마스킹했습니다.

승인 FAQ 49건을 구축하고 검색 평가 질문 68건을 통해 역할별 검색, 낮은 유사도 차단과 복합 질문 처리를 확인했습니다. 개인정보 마스킹과 모델 장애 대응 등을 포함한 전체 AI 자동 테스트 242건이 통과했습니다.

이 경험을 통해 AI 서비스의 품질은 모델의 답변 능력만으로 결정되지 않는다는 것을 배웠습니다. AI가 처리할 부분과 백엔드가 다시 확인할 부분, AI가 실패해도 유지해야 할 기능을 함께 정해야 사용자가 믿고 사용할 수 있는 서비스를 만들 수 있다고 생각합니다.

04 진행 상태를 직접 확인하고 조율한 프로젝트 리더

최종 프로젝트에서 프로젝트 리더(PL)를 맡아 백엔드, 프론트엔드, AI 기능 사이의 연동 문제를 팀원들과 조율했습니다. 프로젝트 중간에 계획보다 실제 작업 인원이 줄어들면서 각자가 맡아야 할 업무가 늘어났고, 공유받은 진행 상황과 실제 구현 상태를 함께 확인할 필요가 생겼습니다.

개발 중에는 기능 간 연동을 막는 문제가 생기면 담당자와 즉시 진행 상태를 확인했습니다. 논의가 길어질 때는 API 계약과 시연에 필요한 핵심 흐름을 먼저 합의하고, 세부 구현은 역할을 나눠 진행한 뒤 함께 검토하자고 제안했습니다.

마감을 앞두고 화면과 API를 기준으로 전체 구현 상태를 점검한 결과, 핵심 사용자 흐름 중 추가 보완이 필요한 부분을 확인했습니다. PM과 남은 기간의 우선순위를 다시 정하고 시연·최종 검증에 필요한 기능을 기준으로 역할을 재조정했습니다. 저도 기존 업무 일정을 조정하며 회원 정보 표시, 고객센터·FAQ 기능, 메일·알림 상태 표시 등 일부 관리자 기능을 보완했고, 제가 구현한 기능과 테스트 결과를 최종 보고서에 정리했습니다.

역할을 조정한 뒤 최종 테스트와 보고서 작성까지 기간 안에 마쳤습니다. 이 경험을 통해 PL은 자신의 업무만 완료하는 것이 아니라, 공유된 진행 상황과 실제 결과물 사이에 차이가 없는지 확인해야 한다는 점을 배웠습니다. 이후에는 중간 결과를 화면과 API로 직접 확인하고, 일정이 늦어지는 부분은 함께 해결할지 역할을 다시 나눌지 더 일찍 판단해야겠다고 생각했습니다.

05 꼼꼼함을 우선순위 판단으로 보완하다

저는 오류를 발견하면 원인뿐 아니라 주변 코드와 전체 흐름까지 확인하려는 편입니다. 병원 전산실에서 근무하며 정상 동작만큼 장애가 발생했을 때의 대응도 중요하다는 것을 배웠기 때문입니다. 다만 모든 내용을 이해한 뒤 다음 단계로 넘어가려다 보니, 한 가지 문제에 예상보다 많은 시간을 쓰기도 했습니다.

FAQ 챗봇을 만들 때도 처음에는 임베딩부터 검색, LLM 연결까지 모든 코드를 순서대로 이해하면서 진행하려 했습니다. 공부에는 도움이 됐지만 이 방식으로는 시간을 맞추기 어렵다고 판단했습니다. 그래서 API의 입력·출력 계약과 사용자 흐름, 반드시 처리해야 할 예외를 먼저 정리한 뒤 핵심 기능을 구현했습니다. 이후 임베딩과 코사인 유사도, 검색 결과가 만들어지는 과정을 다시 검증하며 이해를 보완했습니다.

중간보고 이후에는 첨부문서 기반 자가 자격 진단 기능을 추가했습니다. 자동 테스트와 기본 동작을 확인한 뒤, 실제 데이터 검증까지 개발 일정과 완료 기준에 포함해야 한다는 점을 배웠습니다. 이를 계기로 기능 구현뿐 아니라 다양한 입력 데이터와 예외 상황에서 발생하는 오류를 찾고 보완하는 시간까지 계획에 포함하고 있습니다.

현재는 작업을 핵심 사용자 흐름, 반드시 처리해야 할 예외, 추가 개선 사항으로 나눠 진행하고 있습니다. 영향이 큰 기능과 오류를 먼저 처리하고, 추가 학습이나 개선이 필요한 내용은 별도로 기록합니다. 꼼꼼하게 확인하는 성향은 유지하되, 일정 안에서 무엇을 먼저 확인할지 구분하는 방식으로 보완하고 있습니다.

입사 후에는 신입 개발자로서 기존 서비스가 어떤 사용자와 업무를 위해 만들어졌는지부터 이해하겠습니다. 그 위에서 안정적인 백엔드 기능을 만들고, AI가 필요한 부분과 기존 로직으로 처리하는 것이 나은 부분을 구분할 수 있는 개발자로 성장하겠습니다.

06 서로 다른 강점을 이해하며 협업의 기준을 넓히다

과거에는 협업에서 각자의 업무를 정확히 수행하는 것이 가장 중요하다고 생각했습니다. 병원 전산실에서 새로운 동료와 함께 근무하며 서로의 업무 처리 방식이 달라 의견을 조율해야 하는 경우가 있었습니다.

그러던 중 제가 업무에서 실수했을 때 그 동료가 자신의 일처럼 함께 해결해 주었습니다. 특히 여러 부서와 쌓아온 신뢰를 바탕으로 필요한 협조를 이끌어내는 모습을 보며, 조직에서는 기술적인 처리 능력뿐 아니라 관계를 조율하고 서로의 부족한 부분을 보완하는 역량도 중요하다는 점을 알게 되었습니다.

이후에는 업무 방식의 차이를 먼저 평가하기보다 각자가 잘할 수 있는 역할과 강점을 확인하려고 노력했습니다. 어려움을 겪는 동료에게 먼저 다가가고, 혼자 해결하기 어려운 문제는 주변과 공유하면서 함께 해결하는 태도도 갖게 되었습니다.

이 경험은 최종 프로젝트에서 PL을 맡았을 때도 도움이 되었습니다. 팀원의 진행 상황만 확인하는 것이 아니라 각자가 맡은 기능과 어려움을 함께 살피고, 일정이 늦어지는 부분은 우선순위를 다시 정하거나 역할을 나누는 방식으로 조율했습니다. 협업은 모두가 같은 방식으로 일하는 것이 아니라, 서로 다른 강점을 연결해 공동의 결과를 완성하는 과정이라고 생각합니다.